

Командная строка и Linux

Уверенно перемещаться по файловой системе из командной строки и понимать пути, окружение, права и базовые Linux-команды.

DAY 20

SHELL

command

directory

absolute path

environment variable

ЦЕЛЬ ДНЯ

Что ты сможешь после этого дня

Конкретные действия вместо списка терминов.

- 01 Уверенно перемещаться по файловой системе из командной строки и понимать пути, окружение, права и базовые Linux-команды.
- 02 Объяснить минимум 5 терминов темы своими словами.
- 03 Прочитать показанный пример записи и предсказать результат.
- 04 Самостоятельно решить похожую задачу после разбора.

СВЯЗЬ

Новая тема не появляется из ниоткуда

Связываем сегодняшний шаг с уже построенной картой курса.

ВЧЕРА

Git и совместная работа

Что уже есть в нашей модели.



СЕГОДНЯ

Командная строка и Linux

Вчера: Git и совместная работа. Сегодня мы добавляем следующий слой: командная строка и linux.

ЗАЧЕМ

Зачем мне это?

Каждая тема решает конкретную проблему.

ПРОБЛЕМА → ПОЛЬЗА

Серверы, Docker, Git и многие инструменты разработки управляются командами. Базовая уверенность в shell снимает страх перед Linux.

Вопрос себе

Какую реальную проблему решает сегодняшняя тема?

Ответь до следующего слайда.

Сначала — человеческий образ

Потом обязательно связываем его с настоящей техникой.

БЫТОВОЙ ОБРАЗ

Терминал — текстовый пульт управления компьютером: текущая папка — место, где ты стоишь, путь — маршрут, команда — действие.

ПЕРЕВОД В IT

shell

программа, принимающая и выполняющая текстовые команды

command

указание операционной системе выполнить действие

directory

папка файловой системы

Слой 1: shell • command • directory

Сначала смысл, затем профессиональный термин.

**shell**

программа, принимающая и выполняющая текстовые команды

**command**

указание операционной системе выполнить действие

**directory**

папка файловой системы

Слой 2: absolute path • environment variable • permission

Сначала смысл, затем профессиональный термин.

absolute path

полный путь от корня файловой системы

environment variable

внешняя именованная настройка процесса

permission

правило доступа к файлу или действию

Слой 3: home directory • relative path

Сначала смысл, затем профессиональный термин.

home directory

личная папка пользователя в системе

relative path

путь, отсчитываемый от текущей директории

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Слой 4: pipe • sudo

Сначала смысл, затем профессиональный термин.

pipe

передача вывода одной команды на вход другой

sudo

выполнение команды с повышенными правами

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Как это работает по шагам

Проследи движение слева направо.



Проговори эту цепочку своими словами, не подсматривая в текст.

Первый минимальный пример

Смотрим только на одну основную идею.



day20_step1

```
pwd  
ls  
mkdir practice  
cd practice  
pwd
```

ЧТО ИСКАТЬ

- Где данные?
- Где действие?
- Где результат?
- Что изменится, если заменить одно значение?

Разбираем пример построчно

Код не должен оставаться магической конструкцией.

pwd

Python выполняет эту строку сверху вниз.

ls

Python выполняет эту строку сверху вниз.

mkdir practice

Python выполняет эту строку сверху вниз.

cd practice

Python выполняет эту строку сверху вниз.

pwd

Python выполняет эту строку сверху вниз.

РЕЗУЛЬТАТ

Код → выполнение → результат

Всегда связываем запись с тем, что реально получилось.



до запуска

```
pwd  
ls  
mkdir practice  
cd practice  
pwd
```

РЕЗУЛЬТАТ

Текущий путь → список файлов →
папка → переход внутрь

ИЗМЕНИЛИ

Теперь изменим пример

Сначала предскажи результат. Не запускай код сразу.



changed

```
mkdir -p project/data  
cd project  
touch app.py  
ls
```



**Что появится в
результате?**

Сформулируй ответ словами.

ОТВЕТ

Сверяем прогноз

Не просто смотри ответ — объясни, почему он именно такой.

РЕЗУЛЬТАТ

В project появляется app.py и папка data

ПОЧЕМУ

Python/система выполняет шаги в записанном порядке. Изменённые данные или действие дают новый результат.

Сравни с первым примером: что изменилось?

Как писать неправильно

Ошибка — отдельный учебный материал, а не повод её скрывать.



```
mkdir practice  
cd practise
```

wrong

СТОП

**Что здесь может сломаться
или ввести нас в
заблуждение?**

Сначала найди проблему сам.

Почему возникает проблема

Переводим техническую причину на обычный язык.

Имена `practice` и `practise` отличаются. Shell не исправляет опечатку и не найдёт несуществующую директорию.

Алгоритм отладки: симптом → конкретная строка/шаг → причина → минимальное исправление → повторная проверка.

ИСПРАВЛЕНИЕ

Исправленный вариант

Меняем только то, что действительно было причиной.



fixed

```
mkdir practice  
cd practice
```

ПРАВИЛО

**Не переписывай всё
решение. Найди
минимальное
осмысленное
исправление.**

Что изменилось после исправления?

Ответь без запуска.

- 1 Исчезла ли исходная причина проблемы?
- 2 Какой результат/поведение теперь ожидаем?
- 3 Как проверить это одним маленьким примером?

Сначала увидим полностью разобранный пример

Только после него будет самостоятельная практика.

ЗАДАЧА

Создать структуру project/data, перейти в project и проверить абсолютный путь.

Не начинаем с кода. Сначала переводим задачу на человеческий язык.

КАК ДУМАТЬ

Как рассуждает программист

Разбиваем большую формулировку на маленькие действия.

- 1 Смотрим текущую папку через `pwd`.
- 2 Создаём вложенные директории `mkdir -p`.
- 3 Переходим `cd project`.
- 4 Смотрим содержимое `ls`.
- 5 Снова `pwd` — видим новое положение.

Шаг 1: собираем основу

Не показываем 20 строк сразу — строим решение частями.



solution_part1

```
pwd  
mkdir -p project/data  
cd project
```

Проверь

- Какие данные уже есть?
- Какой шаг плана реализован?
- Что ещё осталось добавить?

ИТОГОВЫЙ КОД

Шаг 2: итоговое решение

Теперь видим программу/запрос целиком.



solution_final

```
pwd  
mkdir -p project/data  
cd project  
ls  
pwd
```

Проверяем результат

Сравниваем то, что ожидали, с тем, что получили.

РЕЗУЛЬТАТ

Путь заканчивается на /project; внутри видна папка data

ГЛАВНОЕ

Задача была решена не потому, что мы «знали готовый код», а потому что сначала построили понятный план.

ПОПРОБУЙ САМ

Практика 1

Задача очень похожа на разобранную — но теперь первый ход за тобой.

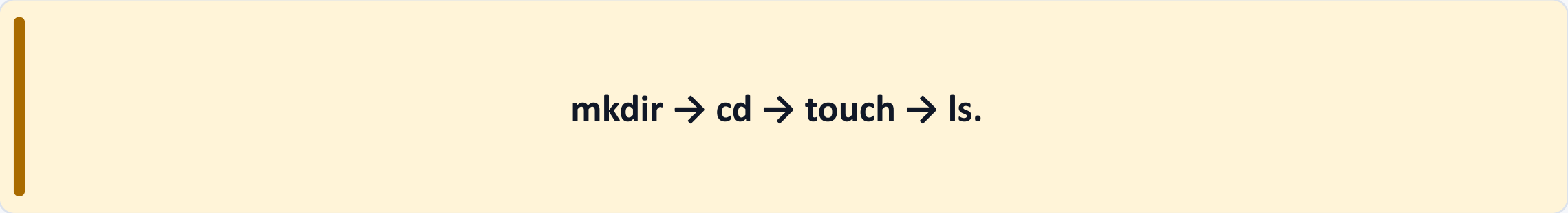
Создай папку `notes`, внутри файл `day20.txt` и выведи список файлов.

Сначала выпиши 2–5 шагов решения словами.

ПОДСКАЗКА

Подсказка — только направление

Попробуй ещё раз до просмотра готового решения.



mkdir → cd → touch → ls.

Вопрос себе: какие конструкции сегодняшнего дня нужны, а какие пока лишние?

Один из правильных вариантов

Сравни логику, а не количество строк.



practice_solution

```
mkdir notes  
cd notes  
touch day20.txt  
ls
```

Ожидаемый результат

Если твой код устроен иначе, но даёт правильное и понятное решение — это нормально.

РЕЗУЛЬТАТ

day20.txt

Сравни

- Результат
- Понятность имён
- Лишние шаги
- Обработку ошибок

Практика 2 — мини-проект

Соединяем несколько элементов дня в одну маленькую закончённую задачу.

ЗАДАНИЕ

Собери структуру `app/{src,data,logs}` одной командой и создай `src/main.py`.

Попробуй сначала без подсказки. Разрешено подсматривать в сегодняшние примеры.

Разбор мини-проекта

Вот один рабочий путь.



```
mkdir -p app/src app/data app/logs  
touch app/src/main.py  
find app -maxdepth 2 -type f -o -type d
```

mini_project

Что должно получиться

Связываем итог с задачей и проверяем смысл.

РЕЗУЛЬТАТ

app, app/src, app/data, app/logs и app/src/main.py

Объясни каждую важную строку/команду своими словами. Если не можешь — вернись к конкретному месту, а не перечитывай весь день.

Четыре ловушки новичка

Лучше увидеть их здесь, чем случайно закрепить как привычку.

01

Удалять/перемещать файлы, не проверив текущую папку.

02

Путать абсолютные и относительные пути.

03

Использовать `sudo` без понимания причины.

04

Вставлять команды из интернета, не читая их.

Проверь себя: 1–2

Не листай дальше, пока не сформулируешь ответ.

- 1 Что такое «shell» и какую проблему это понятие помогает описать?
- 2 Чем «shell» отличается от «absolute path»? Объясни без копирования определения.

Проверь себя: 3–4

Здесь проверяем понимание кода и процесса мышления.

- 3 Какой результат даёт первый минимальный пример и почему?
- 4 Какую ошибку/риск мы разобрали сегодня и какое минимальное исправление нужно?

ОТВЕТЫ

Ответы к вопросам 1–2

Формулировка может отличаться — важен смысл.

1 **shell — программа, принимающая и выполняющая текстовые команды.**

2 shell: программа, принимающая и выполняющая текстовые команды. absolute path: полный путь от корня файловой системы.

ОТВЕТЫ

Ответы к вопросам 3–4

Сверяем причинно-следственную связь.

3 Текущий путь → список файлов → новая папка → переход внутрь

4 Имена practice и practise отличаются. Shell не исправляет опечатку и не найдёт несуществующую директорию.

ИТОГ

Сегодня мы научились

Не перечисляем слова — фиксируем новые способности.

- ✓ Уверенно перемещаться по файловой системе из командной строки и понимать пути, окружение, права и базовые Linux-команды.
- ✓ Читать и объяснять пример темы «Командная строка и Linux».
- ✓ Находить типичную ошибку и формулировать причину.
- ✓ Решать похожую задачу по шагам, а не угадывать готовый код.

ДАЛЬШЕ

Что будет завтра

Связываем сегодняшний результат со следующим шагом.

Базы данных и SQL

Завтра: Базы данных и SQL. Сегодняшняя модель станет опорой для следующего шага.

Сегодняшняя тема не заканчивается здесь — она станет инструментом в следующем дне.